DecisionTreeRegressorModel

December 11, 2025

[ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
import numpy as np
import ast
import matplotlib.pyplot as plt
import seaborn as sns
```

[ ]:
```
# ---- 1. Load cleaned dataset ----
df = pd.read_csv("mxmh_cleaned.csv")

# Convert string list → Python list
df["genre_list"] = df["genre_list"].apply(ast.literal_eval)

# ---- 2. Split features + target ----
X = df.drop(columns=["genre_list"])
y_raw = df["genre_list"]

# ---- 3. Binarize multi-label targets ----
mlb = MultiLabelBinarizer()
y = mlb.fit_transform(y_raw)

genre_labels = mlb.classes_
print("Output genre labels:", genre_labels)

# ---- 4. Train/test split ----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ---- 5. Decision Tree Model ----
tree = DecisionTreeRegressor(max_depth=6, min_samples_leaf=5)
tree.fit(X_train, y_train)

# ---- 6. Predictions ----
y_pred = tree.predict(X_test)
```

```python
# ---- 7. Evaluation ----
rmse_per_genre = {}
r2_per_genre = {}

for i, genre in enumerate(genre_labels):
    rmse = np.sqrt(mean_squared_error(y_test[:, i], y_pred[:, i]))
    r2 = r2_score(y_test[:, i], y_pred[:, i])
    rmse_per_genre[genre] = rmse
    r2_per_genre[genre] = r2

print("\n=== RMSE per genre ===")
for g, v in rmse_per_genre.items():
    print(f"{g}: {v:.4f}")

print("\n=== R² per genre ===")
for g, v in r2_per_genre.items():
    print(f"{g}: {v:.4f}")

print("\nAverage RMSE:", np.mean(list(rmse_per_genre.values())))
print("Average R²:", np.mean(list(r2_per_genre.values())))
```

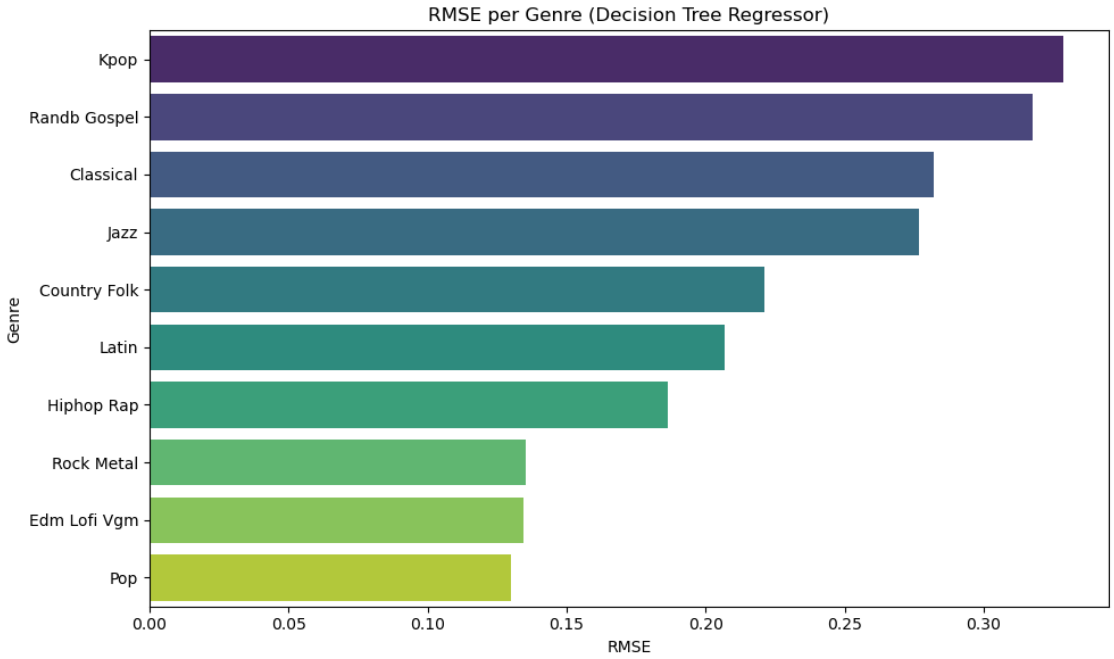
```
Output genre labels: ['Classical' 'Country Folk' 'Edm Lofi Vgm' 'Hiphop Rap'
'Jazz' 'Kpop'
 'Latin' 'Pop' 'Randb Gospel' 'Rock Metal']

=== RMSE per genre ===
Classical: 0.2821
Country Folk: 0.2213
Edm Lofi Vgm: 0.1343
Hiphop Rap: 0.1863
Jazz: 0.2767
Kpop: 0.3285
Latin: 0.2069
Pop: 0.1300
Randb Gospel: 0.3176
Rock Metal: 0.1355

=== R² per genre ===
Classical: 0.4948
Country Folk: 0.7042
Edm Lofi Vgm: 0.9201
Hiphop Rap: 0.7996
Jazz: 0.1058
Kpop: 0.0764
Latin: -0.3111
Pop: 0.9316
Randb Gospel: 0.3239
```

```
Rock Metal: 0.9249

Average RMSE: 0.22192752970508015
Average R²: 0.4970320356872261
```

[ ]:
```python
# Analysis of Model Performance
# 1. Overall Performance Summary
# RMSE = 0.222: on average, predicted probablities deviate from the true␣
 ↪multi-hot labels by around 22 percentage points
# R² = 0.497: the model explains around 50% of the variance in the multi-label␣
 ↪genre data which is moderate performance but uneven across classes
# This imabalance indicates that the model learns some genres very well but␣
 ↪struggles with others
```

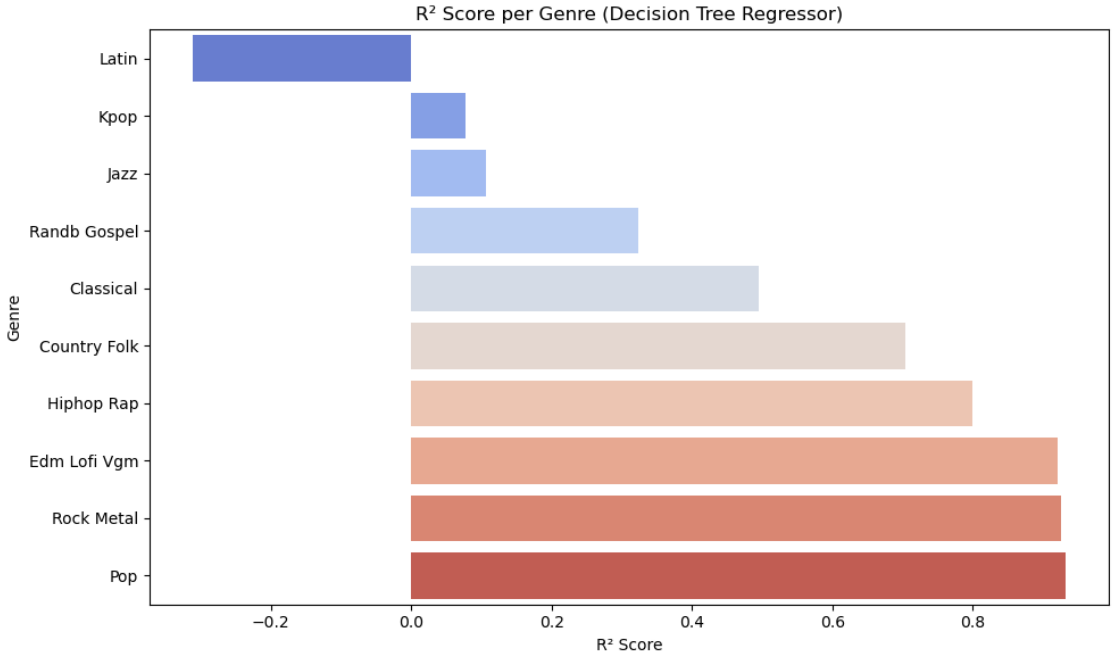
[ ]:
```python
# Visualization: RMSE per genre
# Convert results to pandas Series
rmse_series = pd.Series(rmse_per_genre).sort_values(ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x=rmse_series.values, y=rmse_series.index, palette="viridis")
plt.title("RMSE per Genre (Decision Tree Regressor)")
plt.xlabel("RMSE")
plt.ylabel("Genre")
plt.tight_layout()
plt.show()
```

```
/var/folders/96/7r_kdgpd7qzbjqbpkfp9gf7m0000gp/T/ipykernel_37136/808061996.py:6:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(x=rmse_series.values, y=rmse_series.index, palette="viridis")
```

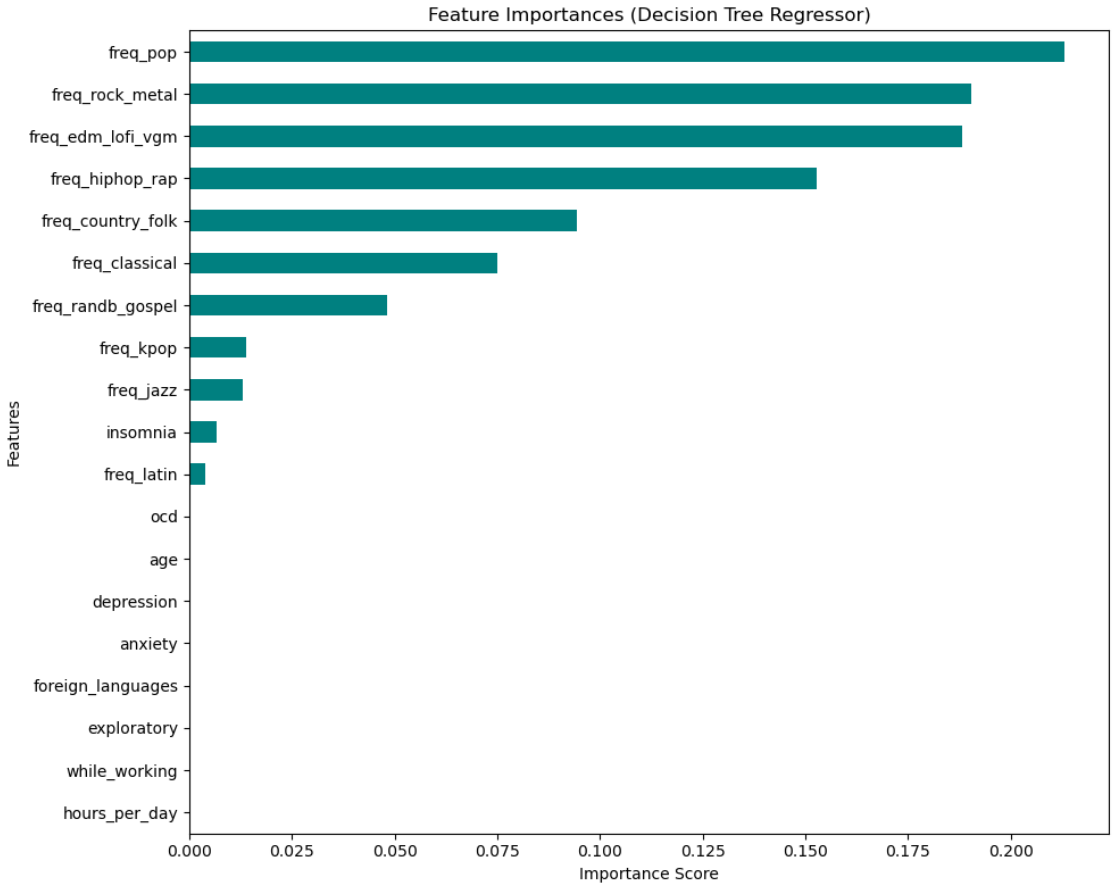


[ ]:
```
# Visualization: R^2 per genre
r2_series = pd.Series(r2_per_genre).sort_values()

plt.figure(figsize=(10, 6))
sns.barplot(x=r2_series.values, y=r2_series.index, palette="coolwarm")
plt.title("R² Score per Genre (Decision Tree Regressor)")
plt.xlabel("R² Score")
plt.ylabel("Genre")
plt.tight_layout()
plt.show()
```

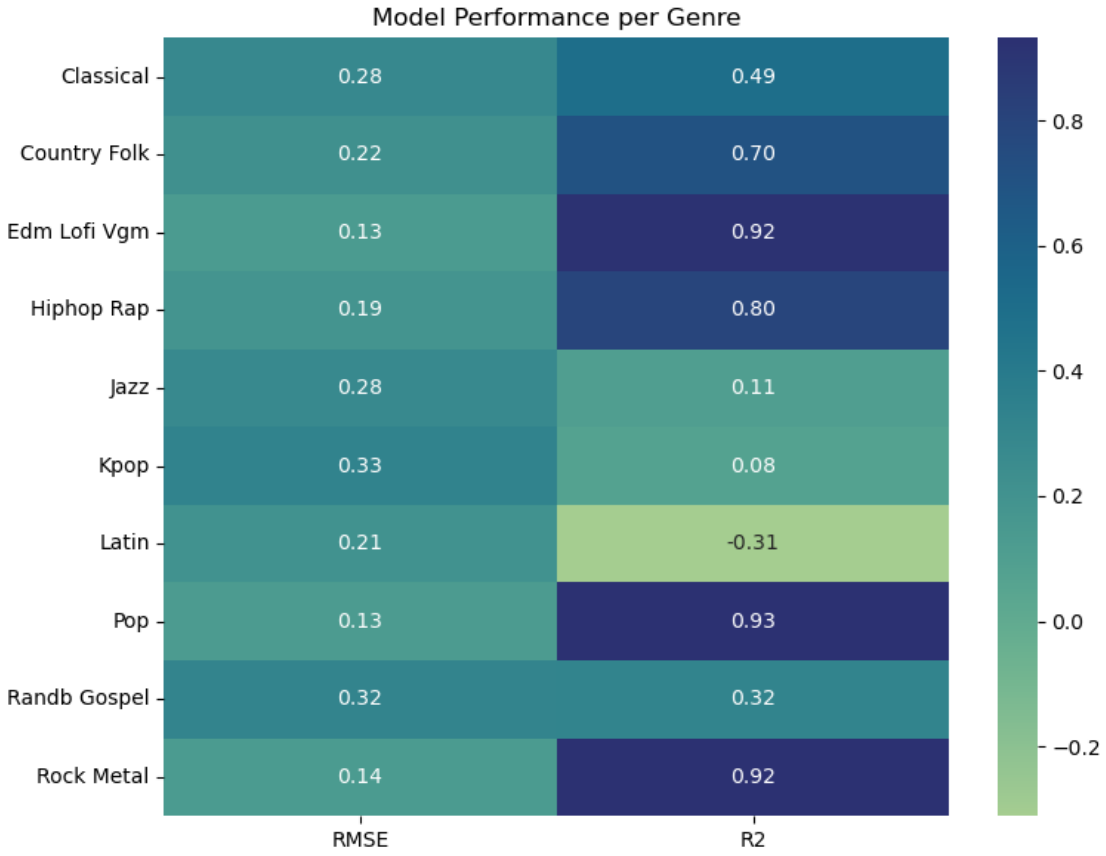
```
/var/folders/96/7r_kdgpd7qzbjqbpkfp9gf7m0000gp/T/ipykernel_37136/625216385.py:5:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(x=r2_series.values, y=r2_series.index, palette="coolwarm")
```



[ ]:
```
# Visualization: Feature Importance (shows which features the model relied on)
importances = pd.Series(
    tree.feature_importances_,
    index=X.columns
).sort_values(ascending=True)

plt.figure(figsize=(10, 8))
importances.plot(kind="barh", color="teal")
plt.title("Feature Importances (Decision Tree Regressor)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()

# which listening habits matter most?
# does mental health scores infuence prediction?
# are there genres that are harder to predict than others?
```



[ ]:
```
# Visualization: Heatmap for Genre Prediction Performance
performance_df = pd.DataFrame({
    "RMSE": rmse_series,
    "R2": r2_series
})

plt.figure(figsize=(8, 6))
sns.heatmap(performance_df, annot=True, cmap="crest", fmt=".2f")
plt.title("Model Performance per Genre")
plt.tight_layout()
plt.show()
```


Model Performance per Genre
Classical
Country Folk
Edm Lofi Vgm
Hiphop Rap
Jazz
Kpop
Latin
Pop
Randb Gospel
Rock Metal
0.28
0.49
0.22
0.70
0.13
0.92
0.19
0.80
0.28
0.11
0.33
0.08
0.21
-0.31
0.13
0.93
0.32
0.32
0.14
0.92
RMSE
R2
0.8
0.6
0.4
0.2
0.0
−0.2